

Register file

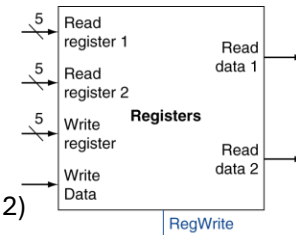
- There are 32 user access registers in our processor.
- Each register can store 32-bit (or 4 bytes) data.
- All register are combined into one structure called Register file

Inputs to the register file:

- Two 5-bits read-register (Read-Register 1 and Read-Register 2)
- One 5-bits Write-Register
- 32-bit Write-Data to register
- 1-bit Register Write (RegWrite) signal

Outputs from register file:

- Two 32-bit which is the content of registers defined by Read-Register 1 and Read-Register 2



There are 32 registers, each of a 32-bit size. All the registers are combined into one structure called a Register File.

Accessing the Register File:

To access (read from or write to) any register in the Register File, we need 5 bits to specify one of the 32 registers. Using 5 bits allows for 32 different possible states, ranging from 00000 to 11111.

Reading from the Register File:

Reading from the Register File means retrieving the content of a specific register to the output bus. For example, to read the content of register 12, we must provide the number of that register (01100) at the “Read register 1” or “Read register 2” input. The Register File will then place the content of register 12 (32 bits) on the output bus (either “Read data 1” or “Read data 2”).

Writing to the Register File:

To write data into the Register File, we send the data to the “Write data” input. We also need to indicate which register to write to by sending the register number to the “Write register” input. Additionally, the control unit will send a “RegWrite” control signal to the Register File to activate the writing process. Once enabled, the data on “Write data” will be stored in the register specified by “Write register”.

Instruction cycle

There are four phases in the instruction cycle, which are:

- **Fetch** the instruction from the instruction memory
- **Decode** the instruction to know what is the operation that should be executed, where are the operands, where the result should go, what is the offset address, ...
- **Execute** the instruction
- **Writeback** the result to the registers, ...

To execute an instruction, the processor follows an instruction cycle, which consists of the steps needed to carry out the instruction. Each instruction cycle has five phases: fetch, decode, execute, and write back.

Fetch Phase:

In the fetch phase, the processor brings the next instruction from the instruction memory. The next instruction to be fetched is indicated by the contents of the program counter. To fetch the instruction, the processor reads the program counter's content, accesses the corresponding location in the instruction memory, and retrieves the instruction stored there.

Decode Phase:

After fetching the instruction from the instruction memory, the next phase is instruction decode. During this phase, the processor determines the function that the instruction will perform, identifies where the operands (the data to be processed) are located, decides where the result should be stored, and classifies the type of the next instruction (whether it is a normal instruction, of branch/jump instruction). All this information becomes available by the end of the decode phase, enabling the processor to execute the instruction in the subsequent phases. Instruction decoding usually occurs in a specialized register called the instruction register, with assistance from the control unit.

Execute Phase:

The third phase, execute, involves performing the instruction. Since the instruction has already been decoded, the processor is now ready to execute it. For example, if the instruction is to add two registers and store the result in a third register, the processor

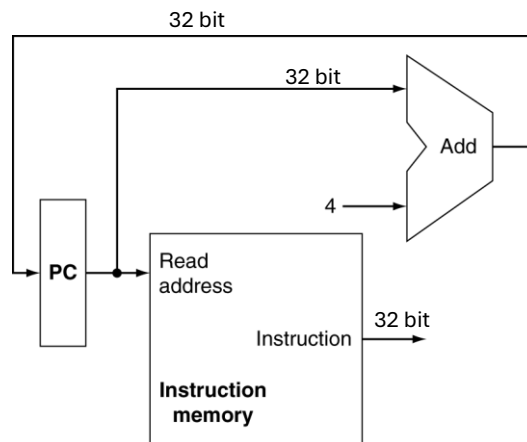
knows which registers to use and what operation to perform. It executes the instruction during this phase of the instruction cycle.

Write Back Phase:

The final phase of the instruction cycle is called write back. In this phase, the processor writes the result back into the register. For instance, if the processor adds two registers together and stores the result in the third register, this result will be written to the destination register during the write back phase. Once the result is written, the execution of the instruction is complete.

Instruction Fetch

- Fetch the instruction from the instruction memory.
- After that the PC is incremented by 4 (assume that current instruction is not jump/branch instruction)



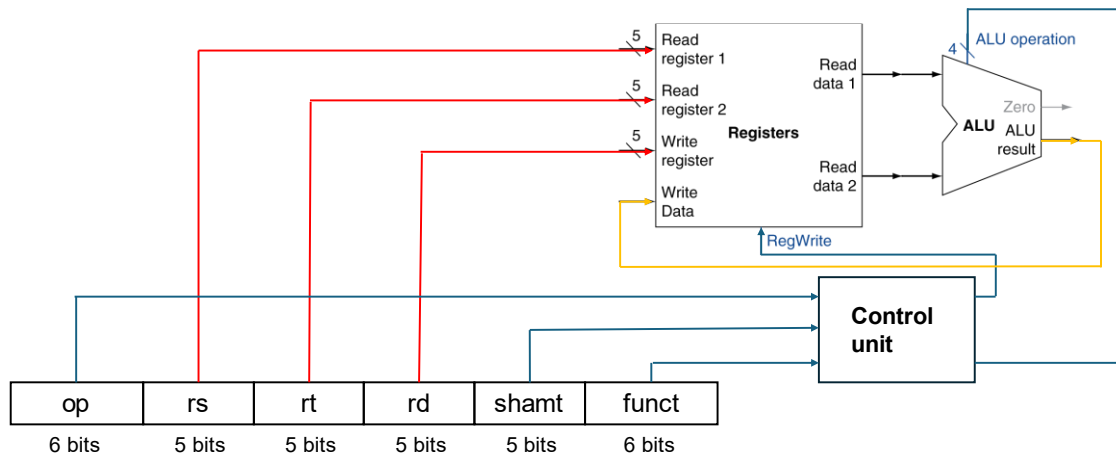
In this phase, the instruction will be fetched from instruction memory. Usually, the address of the next instruction to be fetched can be found on the program counter PC. The program counter provides the address signal used to access instruction memory to retrieve the next instruction. Here, we see that the PC is connected to the read address of that instruction memory in this slide.

After fetching the instruction, the program counter should be updated to point to the next instruction in the instruction memory. If we assume that the current instruction being executed is a normal instruction (i.e., not a branch or jump instruction), then the program counter should be increased by 4. This is because the next instruction to be executed will be the next instruction in the memory, which is 4 bytes higher than the current instruction's address. Thus, the PC is incremented by 4 after fetching an instruction.

In the diagram on the slide, the current value of the program counter is incremented by 4 using the adder (ALU), and the result, shown on the right side of the ALU, is sent back

to the program counter to update its content. This updating process occurs after fetching each instruction from instruction memory. Finally, as you see, the address bus is 32-bit width, therefore all the address buses in the system should be 32-bit width.

Register Format Instructions



Register Format Instructions

- Read two register operands (rs & rt)
- Perform arithmetic/logical operation based on opcode & function
- Write register result (rd)

As you know, there are different types of instruction formats: the register format, the immediate format, and the jump or branch format. In this slide, we are discussing the

execution of the register format instruction. In this format, the source and destination are registers, and they are defined within the instruction itself.

As we can see in the figure, **rs** is the source register one, connected to the read register one. The bus width is 5 bits because the size of the field that defines the register is 5 bits. As mentioned in the previous slide, we need 5 bits to define a register. The 5 bits of **Rs** are used to define **rs** and are connected to the input “Read Register 1” of the register file. Now the content of that register is sent to the output of register file “**Data Register 1**”. At this point, we read **Register 1**.

Similarly, we do the same for the second source register, which is defined in the **rt** field. As shown, the **rt** field is connected to **Read Register 2**.

For the destination register, **rd**, we connect it to the **Write Register** since we are writing to this register, while for the source registers (**rs** and **rt**), we are reading the contents of these registers.

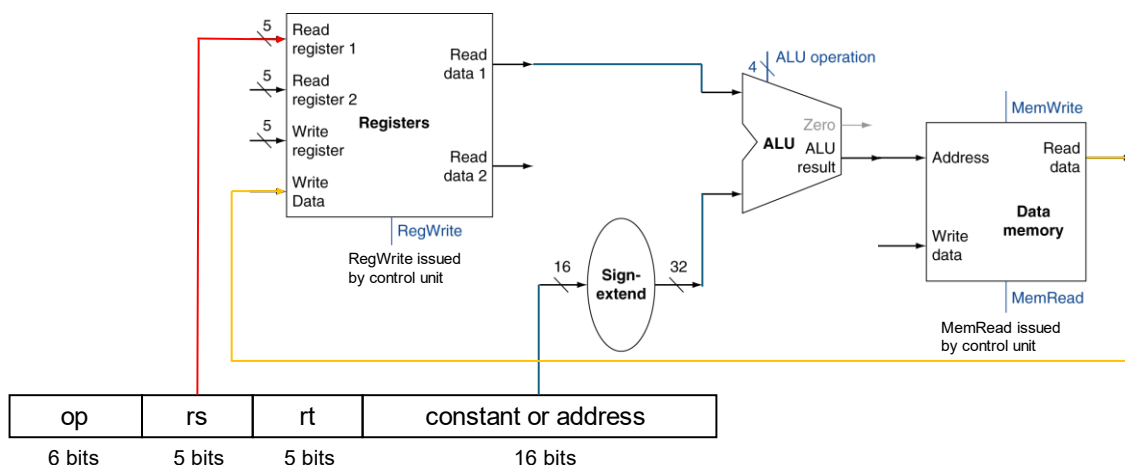
The opcode, function, and shift fields provide important information that the control unit needs to generate control signals to go forward in the executing of the instruction. Once the control unit receives this data, it issues control signals. One of these signals is the **Write Register**, which tells the register file to store the data currently at “Write data” input on the register defined at “Write register”. Another signal that the control unit issues is the **ALU Select**, which selects the operation performed by the ALU.

Once ALU receives the data (which is ready once we define **rs** and **rt**), and **Function Select** signal from control unit, the ALU performs the operation based on the **Function Select** signal from the control unit. The result, appearing at the output of the ALU, is connected back to the **Write Data** input of the register file. This 32-bit result will be stored in the register defined by the **Write Register** (which is based **rd**). When the result is available at the input of the register file, the control unit issues the **Write Register** signal, and the data is written into that register.

Load/Store Instructions

- Read register operands
- Calculate address using 16-bit offset
 - Use ALU, but sign-extend offset
- Load: Read memory and update register
- Store: Write register value to memory

Load/Store Instructions



This slide shows the steps involved in executing load/store instructions. Typically, these instructions use the immediate format, where two registers (**rs** & **rt**) are defined within the instruction, along with the opcode and a 16-bit field that specifies the offset value for the memory address.

The **rs** register is used to define the base address in the memory, which is stored in this register. The base address is 32 bits in size, and it will be added to the offset value defined within the instruction. Since the offset value is 16 bits, it is passed through a sign extension stage to increase its size from 16 to 32 bits. The additional bits in the upper half of 32 bits are filled using the sign of the original 16-bit value, meaning bit #15 (the highest bit of the 16-bit field), making the offset 32 bits in size.

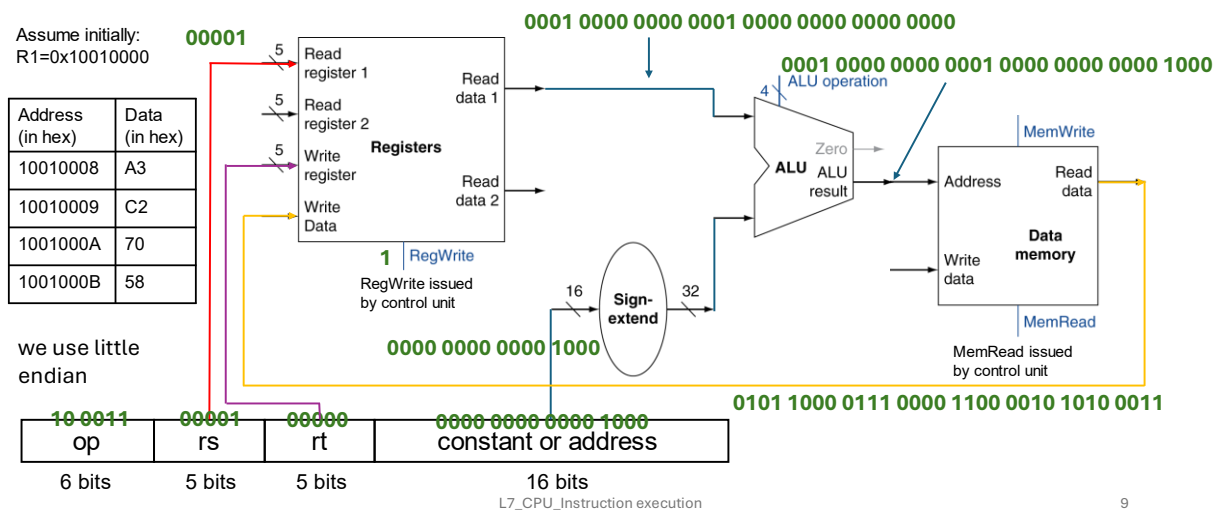
The ALU then adds this 32-bit offset to the base address from the register file defined in **rd** field and appears at the output of the register file **Read Data 1**, producing the final memory address (the effective address that will be accessed). The ALU's output is connected to the address line of the data memory, allowing it to access the correct memory location.

The data will appear once the control unit sends the **Memory Read** signal. Once this signal is issued and the address is defined, the data memory responds by placing the data on its output, **Read Data** (shown as the yellow line in the figure). This 32-bit data is then sent back to the **Write Data** input of the register file to store it in the register defined by **rt**.

As shown, **rt** is connected to the **Write Register** input, and the control unit will issue a control signal to write this data into the register specified by **rt**. Once this process is complete, the load instruction has been executed, and the data is successfully fetched from memory and stored in the register defined by **rt**.

Load/Store Instructions, example:

lw R0, 8(R1)



This slide shows an example of executing a load instruction. The instruction is a load register **R0** with the data from memory location defined by the effective address generated by adding an offset to the content of register **R1**.

The instruction set architecture (ISA) of this assembly instruction is shown, with all the fields filled in based on the instruction. We have **rs** as register **R1** (represented as 00001), **rt** as register **R0** (represented as 00000), and an offset address of **8**. The opcode for this load instruction is 100011, based on the instruction set.

Each field is connected to its appropriate place. The **rs** field is connected to the **Read Register 1** input because **rs** defines the content of **R1**, which stores the base address. In this case, **R1** corresponds to register 1, so **Read Register 1** receives the value **1** as input. The content of **R1** will then appear at the **Read Data 1** output, which is connected to the input of the ALU. Now the ALU receives the base address at its first input.

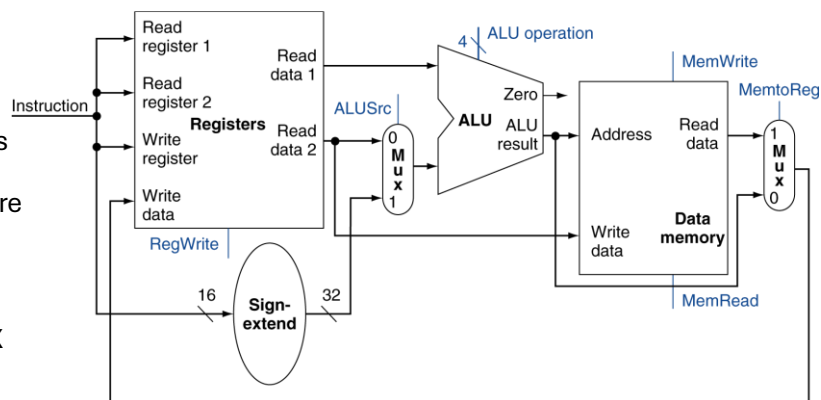
The second input of the ALU receives the offset value, which is extended from 16 bits to 32 bits by sign extension. The sign extension fills the upper 16 bits with the sign of the most significant bit (bit 15) of the original 16-bit value. In this case, bit 15 is 0, so all the new upper 16 bits will be zeros.

Now, the ALU adds the base address from **R1** to the offset, generating the effective address. This effective address is connected to the address line of the data memory. The control unit then issues the **Memory Read** signal. The memory responds by sending the contents of the memory location defined by the effective address, the memory will place that data at its output **Read data**.

In our example, the data returned from memory is **0x5870C2A3** (in hexadecimal format), which is sent back to the register file via the **Write Data** input. The destination register is **R0**, and the control unit will also be issued by the control unit and sent to the **RegWrite** input in the register file. The register file will then write the 32-bit data into the register defined by the **Write Register** input, which in this case is **R0**.

R-Type/Load/Store Datapath

- The system shown is used with both R-format and Load/store format instructions.
- The main difference here is adding the multiplexer **2:1 MUX**



R-format instruction → ALU Src = 0

Branch/jump-format instruction → ALU Src = 1

The system shown is used with both R-format and load/store format instructions. The main difference is the addition of a 2:1 multiplexer (MUX). This multiplexer has a select signal that determines which of the input channels is sent to the MUX output. The select signal is **ALU Src**, which is issued by the control unit based on the current instruction.

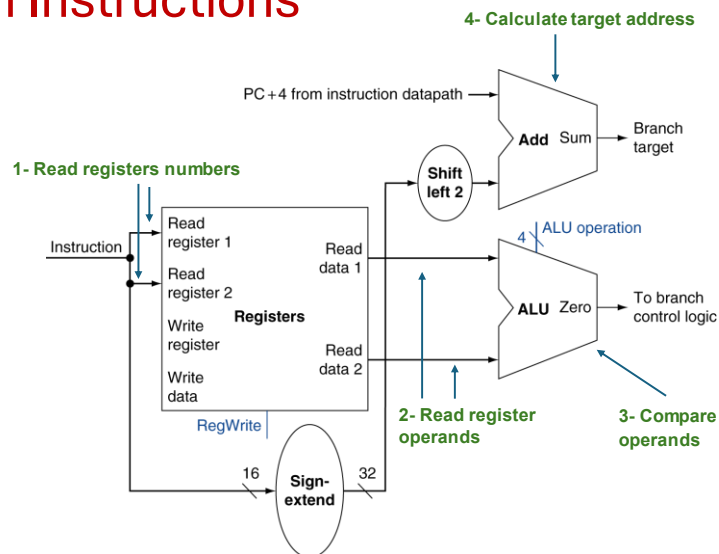
If the current instruction is an R-format instruction, the control unit makes **ALU Src** equal to 0, causing the MUX to pass its first input to the output (which is also the second input to the ALU). In an R-format instruction, the second input to the ALU comes from the register file's **Read Data 2**, because in this format, both operand sources—**rs** (source 1) and **rt** (source 2)—are defined within the instruction, and there is no offset value.

On the other hand, if the current instruction is a branch/jump instruction, the second input to the ALU should come from the output of the sign extension unit, as branch/jump instructions include a 16-bit offset address. This offset is used by ALU. Therefore, when the current instruction is a branch/jump instruction, the control unit sets **ALU Src** to 1, causing the MUX to pass its second input to the output (which is also the second input to the ALU).

Branch Instructions

- Read register operands
- Compare operands
 - Use ALU, subtract and check Zero output
- Calculate target address
 - Sign-extend displacement
 - Shift left 2 places (word displacement)
 - Add to PC + 4
 - Already calculated by instruction fetch

Branch Instructions



Here, we have branch instruction, and as you know, branch instructions compare the contents of two registers, **rt** and **rs**. Based on the result, the instruction may or may not be executed. There are two types of branch instructions: **branch on zero (BEQ)** and **branch on not equal to zero (BNE)**.

- **Branch on zero (BEQ)** is executed if the contents of the two registers are equal; otherwise, it is not executed.
- **Branch on not equal to zero (BNE)** is taken when the contents of the two registers differ; if they are the same, the instruction is not executed.

In general, there are two possible outcomes:

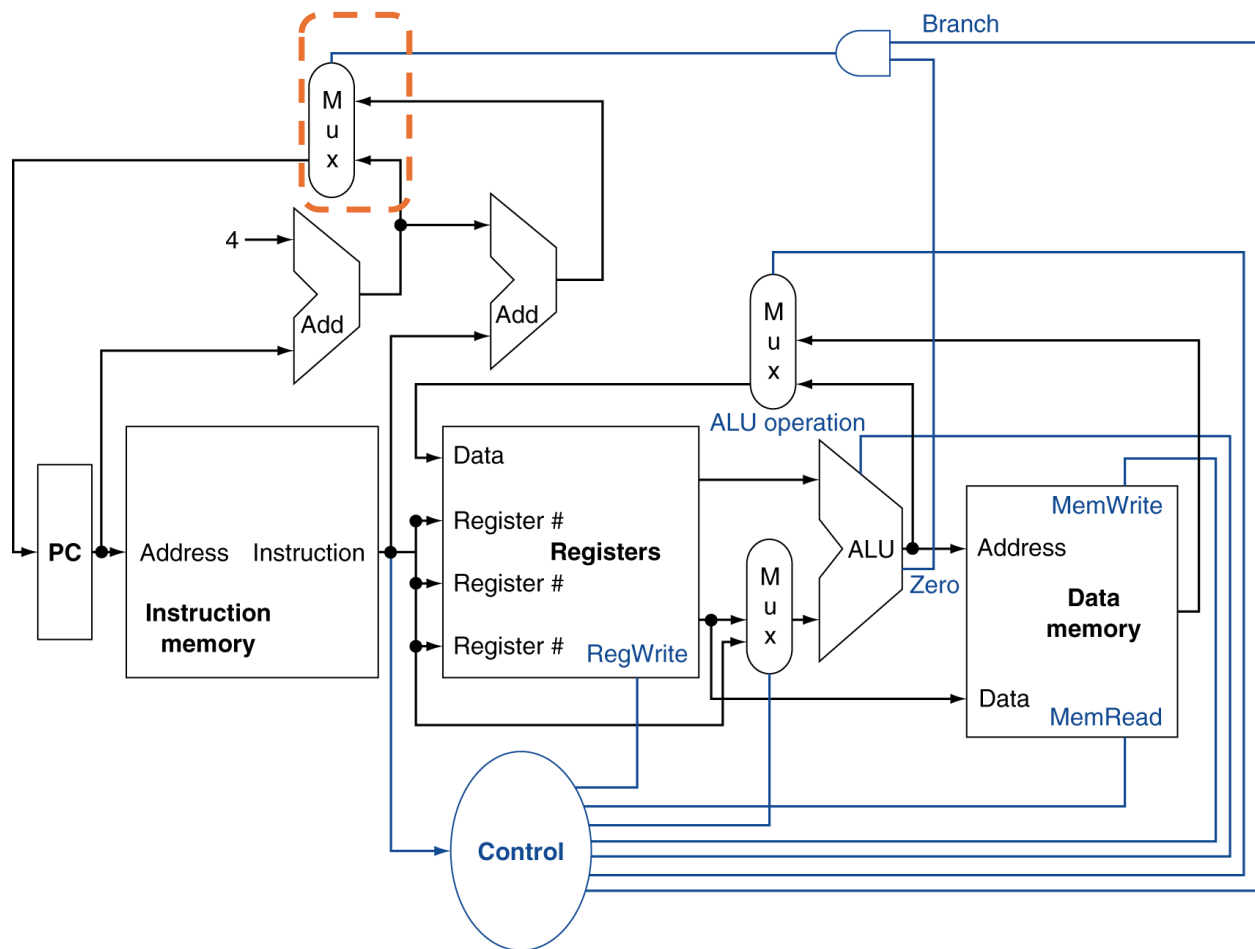
1. **Branch is taken:** In this case, the branch address is calculated as the value inside the program counter (PC) plus the offset address defined in the instruction.
2. **Branch is not taken:** In this case, the next instruction to be executed is the next sequential instruction in the program and it is stored at PC+4

As seen in the diagram, the instruction is connected to the **Read Register 1** and **Read Register 2** inputs of the register file, as well as to the sign extension unit. Once the register file receives the signals, it outputs the contents of the two registers on the **Read Data 1** and **Read Data 2** lines. These outputs are connected to the ALU inputs. The ALU subtracts **Read Data 2** from **Read Data 1**, and based on the result, the ALU's **Zero Flag** is updated. The **Zero Flag** will be set to 1 if the result is 0, and 0 if the result is non-zero.

At the same time, the sign extension unit extends the 16-bit offset by adding 16 bits to the left, filling the new bits with the sign of the 16-bit value (bit #15). This produces a 32-bit offset, which is then sent to the **Shift Left 2** unit. The reason for shifting left 2-bits is that each instruction is aligned to four memory locations (word alignment), so the shift ensures the address is correctly aligned.

The upper ALU on the diagram computes the next program counter value by (PC+4) to the 32-bit offset value. The result of the ALU (the branch target address) is then sent to the program counter if the branch condition is true.

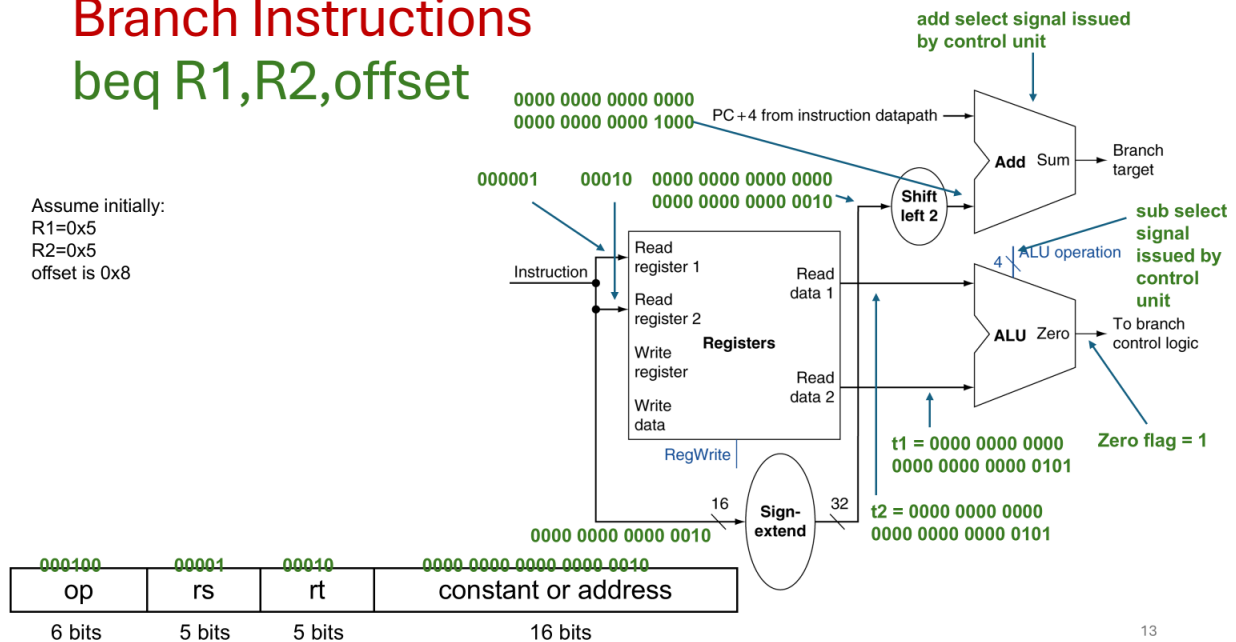
If the branch condition is false, then the new value of PC is PC + 4 instead of the branch target address. The control signal from the control unit, along with the **Zero Flag** from the ALU will be issued. This control signal determines which address is selected by generating a select signal of 2:1 MUX that is used to select the next value of the PC which is either the MUX—the branch target or the next sequential address. See slide below.



Branch Instructions

beq R1,R2,offset

Assume initially:
R1=0x5
R2=0x5
offset is 0x8



This slide shows the execution of a **branch on equal (BEQ)** instruction. As shown above, the instruction compares the contents of registers **R1** and **R2**. If they hold the same values, the branch is taken; otherwise, it is not.

In the instruction, the fields are filled as follows:

- **rs** is register **R1** (which is represented as 1),
- **rt** is register **R2** (which is represented as 2),
- The 16-bit **offset** is provided, and here we assume it is **0002**, after shifting right the offset (which is 8) 2-bits
- The **opcode** for the branch instruction is **4**, which can be found in the instruction set.

At the output of the register file, we get the contents of **Read Data 1** and **Read Data 2**, which correspond to the contents of **R1** and **R2**. The lower ALU subtracts the value of **R2** from **R1**, and in this case, the result is 0 because both registers contain the same value (5). Since the result is 0, the **Zero Flag** will be set to 1, indicating that the branch should be taken.

Next, the effective address or branch address needs to be computed. The branch address is calculated as the current value of the program counter (PC) plus 4 (this is the input of the upper ALU) plus the 16-bit offset (which has been extended to 32 bits using

sign extension and then shifted left by two to account for word alignment). Word alignment is used to provide more data range for the 16-bit offset field in the instruction.

The upper ALU adds these two values ($PC + 4$) and the shifted offset value to create the branch address. This branch address will then be used to update the value of the program counter, and the next instruction executed will be the one located at this new branch address.

Full Datapath

